ChessBase

A Chess Game Database & Move-Based Search Platform

Project Synopsis

DBMS Experiential Learning – Level 3 Project

Team Member	SRN
Raghav Budur	PES1UG23AM225
Prithviraj K	PES1UG24AM204

1. Problem Statement

Millions of chess games are freely available online, yet finding games that are relevant to a specific strategic idea, tactical pattern, or move sequence is difficult using conventional keyword search or manual database browsing. Existing chess database tools index games mainly by player, opening code, and result – they are not built to retrieve games by the actual content of the moves played, such as *"games with a kingside pawn storm against the Sicilian Dragon"* or *"games with a queen sacrifice leading to a mating attack"*. There is a need for a database platform that stores games and moves precisely in a relational schema, while also indexing move sequences and engine/opening annotations so that games can be searched by the strategic and tactical content of their moves, not just by metadata.

2. Objectives

- Design and implement a normalized relational schema to store players, tournaments, games, openings, and move-by-move data.
- Index move sequences, engine annotations, and opening descriptions so that games can be searched by tactical/strategic content, not just by player or result.
- Enable move-based and similarity search over games (e.g. by strategic motif or move pattern) using vector similarity search alongside precise SQL filtering.
- Demonstrate at least four advanced database features: stored procedures, triggers, views, indexing, full-text search, and vector similarity search.
- Build a functional Next.js web application exposing a player dashboard, game explorer with PGN/move viewer, opening explorer, and move-based search interface.
- Implement role-based authentication distinguishing Admin (data import/management) and Player/User (search and analysis) capabilities.

3. Scope

The platform covers ingestion of PGN-format game datasets, extraction of structured metadata (players, tournaments, openings, moves, results), generation of textual descriptions/annotations for games and openings, and indexing of move sequences and engine/opening annotations for pattern-based search. The scope includes a web interface for browsing, searching, and visually replaying games, along with basic analytics (player statistics, opening success rates). Real-time chess engine evaluation, live multiplayer play, and mobile clients are out of scope for this iteration and are listed under future enhancements.

4. Functional Requirements

- **FR1 – Data Import:** Admin can bulk-import PGN game datasets; the system parses and populates players, tournaments, games, openings, and moves.
- **FR2 – Structured Search:** Users can filter/search games by player, opening (ECO code), tournament, date range, and result.
- **FR3 – Move/Pattern Search:** Users can search by strategic or tactical motifs (e.g. move patterns, sacrifices, attack types) and retrieve the most relevant games via vector similarity search over move and annotation data.
- **FR4 – Game Explorer:** Users can view a game's PGN, step through moves on a board, and read associated annotations.

- **FR5 – Player Dashboard:** Users can view a player's rating history, win/loss/draw statistics, and most successful openings.
- **FR6 – Opening Explorer:** Users can browse an opening tree with move-by-move success percentages for each side.
- **FR7 – Favourites:** Authenticated users can save games/players to a personal favourites list.
- **FR8 – Authentication & Roles:** Admin and Player/User roles have distinct permissions enforced at the API layer.

5. System Architecture

The system follows a three-tier architecture. The **presentation tier** is a Next.js application (React Server Components + API/Route Handlers) providing the dashboard, explorer, and search UIs. The **application/API tier** exposes REST endpoints that route structured queries to PostgreSQL and move-pattern queries to the vector store, and orchestrates hybrid queries that combine both (e.g. "Sicilian Dragon games with a kingside attack, sorted by rating"). The **data tier** consists of two database paradigms working together, as justified below.

Layer	Technology	Responsibility
Frontend	Next.js (App Router), React, Tailwind	Dashboard, Game/Opening Explorer, Move/Pattern Search UI, chess board rendering
Backend / API	Next.js API Routes (Node.js)	Auth, request validation, query orchestration between SQL and vector store
SQL Database	PostgreSQL	Players, tournaments, games, openings, moves, ratings – all structured, relational data
Vector Database	PostgreSQL + pgvector (or Qdrant)	Embeddings of move sequences, opening descriptions, and engine annotations for move-pattern search
Auth	NextAuth.js / JWT	Role-based access for Admin and Player/User

Query Flow – Hybrid Search Example

For a query such as *"queen sacrifice leading to a kingside mating attack"*: (1) the query is embedded using the same embedding model used at ingestion time over move sequences and annotations, (2) a cosine-similarity search (`ORDER BY embedding <=> query_embedding`) retrieves the top-N matching game/annotation vectors from the vector store, (3) the returned game IDs are joined back against PostgreSQL to fetch full structured details (players, result, opening, tournament) for display.

6. Database Choice and Justification

This project follows **Option 2: SQL + Vector Database**, since chess data naturally splits into two categories that are best served by two different paradigms.

6.1 SQL (PostgreSQL) – for structured data

Players, tournaments, games, openings, and moves have fixed, well-defined attributes with clear relationships (a game references two players, one tournament, and one opening). This is a textbook case for a relational model: it lets us enforce primary/foreign-key integrity, run precise filters and aggregations (JOIN, GROUP BY, HAVING), and guarantee consistency – none of which a document or vector store does as naturally.

6.2 Vector Database (pgvector) – for unstructured/semi-structured data

PGN annotations, engine commentary, opening descriptions, and free-text game summaries do not have a fixed schema and cannot be meaningfully queried with exact-match SQL – there is no WHERE clause for "aggressive style" or "kingside attack." Embedding this text and retrieving by cosine similarity is the natural fit, and is why the vector database is justified here rather than included merely for technology demonstration.

6.3 Why not NoSQL instead?

A document store (e.g. MongoDB) would not add retrieval capability beyond what PostgreSQL's JSONB columns already offer for this project's semi-structured fields, and would not solve the core problem – move-pattern retrieval over unstructured annotation and move-sequence data – which specifically requires vector similarity search. Hence SQL + Vector DB was chosen over SQL + NoSQL.

7. ER Diagram (Structured Description)

The relational model below is expressed as an entity-relationship description; a graphical ER diagram (crow's-foot notation) should be produced from this in the full report using a tool such as dbdiagram.io or draw.io. Entities and relationships:

Entity	Key Attributes	Relationships
Player	player_id (PK), name, fide_rating, country	1 — N Game (as White), 1 — N Game (as Black)
Tournament	tournament_id (PK), name, location, start_date	1 — N Game
Opening	opening_id (PK), eco_code, opening_name	1 — N Game
Game	game_id (PK), white_player_id (FK), black_player_id (FK), tournament_id (FK), opening_id (FK), result	N — 1 Player (x2), Tournament, Opening; 1 — N Move
Move	move_id (PK), game_id (FK), move_number, notation	N — 1 Game

Cardinality summary: One Player participates in many Games (as White or Black); one Tournament hosts many Games; one Opening is played in many Games; one Game consists of many Moves. All foreign keys in `Game` and `Move` enforce referential integrity against their parent tables.

8. Relational Schema (DDL)

```
CREATE TABLE Player (
    player_id     SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    fide_rating   INT CHECK (fide_rating >= 0),
    country       VARCHAR(60)
);

CREATE TABLE Tournament (
    tournament_id SERIAL PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    location      VARCHAR(100),
    start_date    DATE
);

CREATE TABLE Opening (
    opening_id    SERIAL PRIMARY KEY,
    eco_code      VARCHAR(3) NOT NULL,
    opening_name  VARCHAR(150) NOT NULL
);

CREATE TABLE Game (
    game_id         SERIAL PRIMARY KEY,
    white_player_id INT REFERENCES Player(player_id),
    black_player_id INT REFERENCES Player(player_id),
    tournament_id   INT REFERENCES Tournament(tournament_id),
    opening_id      INT REFERENCES Opening(opening_id),
    result          VARCHAR(7) CHECK (result IN ('1-0','0-1','1/2-1/2')),
    CHECK (white_player_id <> black_player_id)
);

CREATE TABLE Move (
    move_id     SERIAL PRIMARY KEY,
    game_id     INT REFERENCES Game(game_id) ON DELETE CASCADE,
    move_number INT NOT NULL,
    notation    VARCHAR(10) NOT NULL
);
```

9. Data Dictionary

Table.Column	Type	Constraint	Description
Player.player_id	SERIAL	PK	Unique player identifier
Player.fide_rating	INT	CHECK ≥0	Current FIDE rating
Tournament.tournament_id	SERIAL	PK	Unique tournament identifier
Opening.eco_code	VARCHAR(3)	NOT NULL	Encyclopedia of Chess Openings code, e.g. B90
Game.white_player_id	INT	FK → Player	Player with white pieces
Game.black_player_id	INT	FK → Player	Player with black pieces
Game.result	VARCHAR(7)	CHECK enum	1-0 / 0-1 / 1/2-1/2
Move.notation	VARCHAR(10)	NOT NULL	Algebraic notation, e.g. Nf3, Qxd8+
game_embeddings.embedding	VECTOR(1536)	NOT NULL	Embedding of game summary / annotation text

10. Vector Database Data Model

Aspect	Detail
Data embedded	Opening descriptions, engine-analysis comments, full-game strategic summaries
Embedding model	Text/sequence embedding model over move notation and annotation text (e.g. 1536-dim)
Storage	PostgreSQL pgvector extension – a `vector(1536)` column linked to game_id / opening_id
Similarity metric	Cosine distance (`<=>` operator in pgvector)
Retrieval	Top-K nearest neighbours to the query embedding, joined back to SQL tables for display

11. Important Queries

11.1 Complex SQL – most successful opening against the Sicilian Defense

```
SELECT o.opening_name, COUNT(*) AS games_won
FROM Game g
JOIN Opening o ON g.opening_id = o.opening_id
WHERE o.opening_name ILIKE '%Sicilian%' AND g.result = '1-0'
GROUP BY o.opening_name
ORDER BY games_won DESC
LIMIT 5;
```

11.2 Top-performing players in a tournament

```
SELECT p.name,
       COUNT(*) FILTER (WHERE g.result='1-0' AND g.white_player_id=p.player_id
                        OR g.result='0-1' AND g.black_player_id=p.player_id) AS wins
FROM Player p
JOIN Game g ON p.player_id IN (g.white_player_id, g.black_player_id)
WHERE g.tournament_id = $1
GROUP BY p.name
ORDER BY wins DESC;
```

11.3 Vector similarity search – move/pattern-based game search

```
SELECT g.game_id, ge.summary_text,
       ge.embedding <=> $1::vector AS distance
FROM game_embeddings ge
JOIN Game g ON g.game_id = ge.game_id
ORDER BY distance
LIMIT 10;
```

12. Advanced Database Features

#	Feature	Application in this Project
1	Stored Procedure	get_player_stats(player_id) – returns wins/losses/draws and rating trend
2	Trigger	After INSERT on Game – auto-updates player win/loss/draw counters
3	View	top_players – ranks players by win rate across all recorded games
4	Indexing	B-tree index on Game.opening_id and Game.tournament_id for fast filtering
5	Full-Text Search	GIN/tsvector index on Opening.opening_name for fuzzy name search
6	Vector Similarity Search	pgvector cosine search over game/opening embeddings

This satisfies the requirement of demonstrating at least four advanced database concepts, with six implemented in total.

13. Roles and Authentication

Role	Permissions
Admin	Import PGN datasets, manage tournaments and openings, moderate content
Player / User	Search and browse games, run move/pattern queries, view analytics, save favourites

Authentication is implemented via NextAuth.js / JWT, satisfying the minimum-two-roles requirement. Database credentials are stored in environment variables and never hard-coded; passwords are hashed (bcrypt) rather than stored in plain text.

14. Web Application (Next.js) Features

- **Player Dashboard** – rating history chart, win/loss/draw statistics.
- **Game Explorer** – interactive PGN viewer with move-by-move navigation and board replay.
- **Opening Explorer** – opening tree with per-move success percentages.
- **Move/Pattern Search** – search bar for tactical/strategic motifs (e.g. "queen sacrifice leading to mate") returning ranked, relevant games.

15. Testing Plan

Test Type	Example Case	Expected Result
Schema integrity	Insert a Game with an invalid opening_id	FK constraint violation, insert rejected
Trigger correctness	Insert a completed Game	Player win/loss counters update automatically
Query correctness	Run top-opening-vs-Sicilian query on seed data	Matches manually verified counts
Vector search relevance	Query "kingside attack"	Top-10 results relevant to the move pattern on manual review
Auth / RBAC	Non-admin calls the import endpoint	403 Forbidden returned

16. Requirement Coverage Summary

Requirement	Covered
SQL Database (PostgreSQL)	Yes
Vector Database (pgvector)	Yes
ER Diagram	Yes
Relational Schema	Yes
Complex Queries (JOIN/GROUP BY/subqueries)	Yes
4+ Advanced DB Features	Yes (6 implemented)
Role-Based Authentication (2+ roles)	Yes
Next.js Web Application	Yes

Requirement	Covered
Real-World Problem, Beyond CRUD	Yes

17. Conclusion and Future Enhancements

The proposed platform demonstrates a real-world justified use of both relational and vector database paradigms: PostgreSQL handles the precise, structured side of chess data (players, tournaments, results, openings), while pgvector enables move-pattern retrieval over unstructured annotations and move sequences that no amount of SQL filtering alone could support. Together with six advanced database features and a full Next.js application, the project goes well beyond CRUD and satisfies the Level-3 evaluation criteria.

Future Enhancements

- Integrate a live chess engine (e.g. Stockfish via UCI) to auto-generate tactical annotations for imported games.
- Add a graph database (Neo4j) layer to model opening transposition trees and player head-to-head networks.
- Support real-time collaborative game analysis and commentary.
- Extend move-pattern search with embeddings that incorporate board-position data, not just move text.

Prepared by: Raghav Budur (PES1UG23AM225) & Prithviraj K (PES1UG24AM204)